[1] ROLE: You are an expert C++17 developer for the **SOLEX** logical query rewriter. Given a formal rule specification $r^* = (\phi_{r^*}, \tau_{r^*}, \mathcal{K}_{r^*})$, implement the corresponding Rule subclass with Matches() and Apply().

[2] RULE API:

```
namespace solex::rules {
 class Rule {
  // Matches(): returns true iff φ_{r*} holds on op; called on every node
  virtual bool Matches(const ir::LogicalOperator& op) const = 0;

  // Apply(): transforms plan per τ_{r*}; return std::move(op) unchanged if inapplicable
  virtual ir::LogicalOperatorPtr Apply(ir::LogicalOperatorPtr op) const = 0;
 }; }

// Key types: LogicalOperatorType::{AGGREGATE | JOIN | SCAN | FILTER | WITH | PROJECT …}
// Key types: JoinType::{INNER | LEFT | RIGHT | FULL | CROSS …}
// Accessors: op.type() op.Cast<T>() op.children() op.Clone()
// AggregateOperator: .group_by() .aggregates()
// JoinOperator: .join_type() .predicates()
// ScanOperator: .table_name() .alias()
```

[3] ϕ_{r^*} Applicability Condition:

- **Root**: AGG[COUNT(*)] with empty GROUP BY (no grouping columns)
- **Subtree**: ≥ 3 scan nodes connected by INNER JOINs forming a star
- **FK structure**: all inter-table join predicates are equalities $T_i.fk_i = T_{anc}.pk$ for a single anchor table T_{anc}
- Every non-anchor T_i links directly to T_{anc} (star topology; no transitive or hub-bypass joins)

[4] τ_{r^*} Rewrite Transformation:

- For each non-anchor branch T_i: build CTE_i = AGG[COUNT(*) GROUP BY fk_i] over SCAN[T_i] + per-table filters
- Join T_{anc} to every CTE_i via $T_{anc}.pk = \text{CTE}_i.fk_i$ (INNER)
- Replace root with: AGG[SUM($b_1.n \times \cdots \times b_N.n$)]

[5] $\mathcal{K}_{r^*}$ Fragment Keys: ■ $\mathcal{K}_1$ AGG(FILTER) ■ $\mathcal{K}_2$ AGG(FILTER(JOIN[INNER])) ■ $\mathcal{K}_3$ AGG(JOIN[INNER]) ■ $\mathcal{K}_4$ AGG(FILTER(JOIN[INNER](JOIN,SCAN))) ■ $\mathcal{K}_5$ AGG(JOIN[INNER](JOIN,SCAN))

[6] Example: Distillation Seed from Which r^* Was Extracted:

<Original SQL Query (Q)>, <Rewritten SQL Query (Q'')>

[7] Task: Implement Matches() and Apply() for the rule above. Matches() must return True if and only if the operator satisfies ϕ_{r^*} exactly as specified. Apply() must produce the rewritten plan according to τ_{r^*} and return a valid LogicalOperatorPtr. **Output only compilable C++17 code** for the two method bodies. No explanation. No markdown fences.